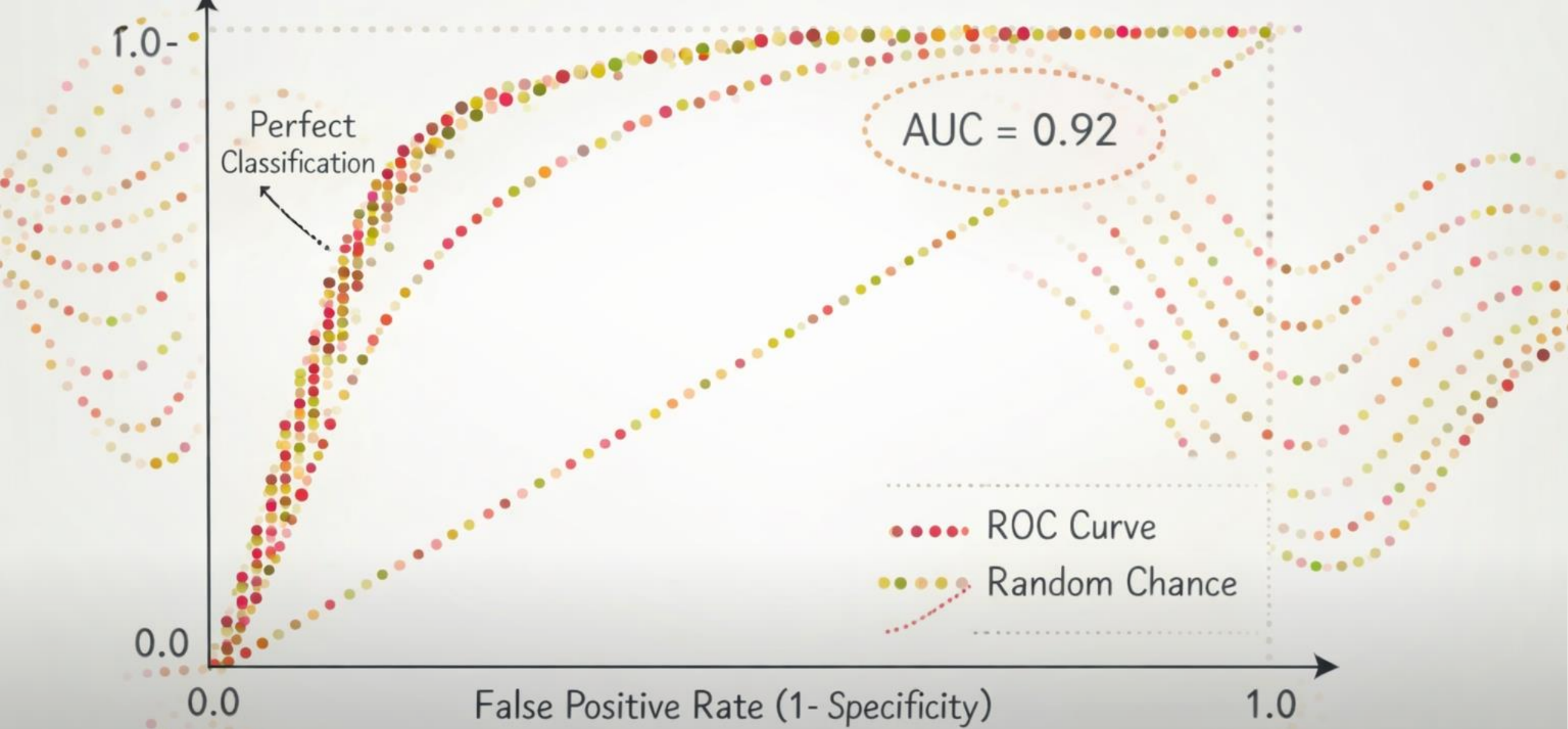


CLASSIFICATION METRICS

REGRESSION VS CLASSIFICATION (REVIEW)

- Regression and Classification are both supervised learning methods
- Regression
  - The target or response variable (y) is numeric
  - Model predicts the *value* of y
  - Prediction accuracy is measured by how close the predicted value is to the truth, such as

$$(y_i - \hat{y}_i)^2 \quad \text{or} \quad |y_i - \hat{y}_i|$$

ACCURACY AND ERROR RATE

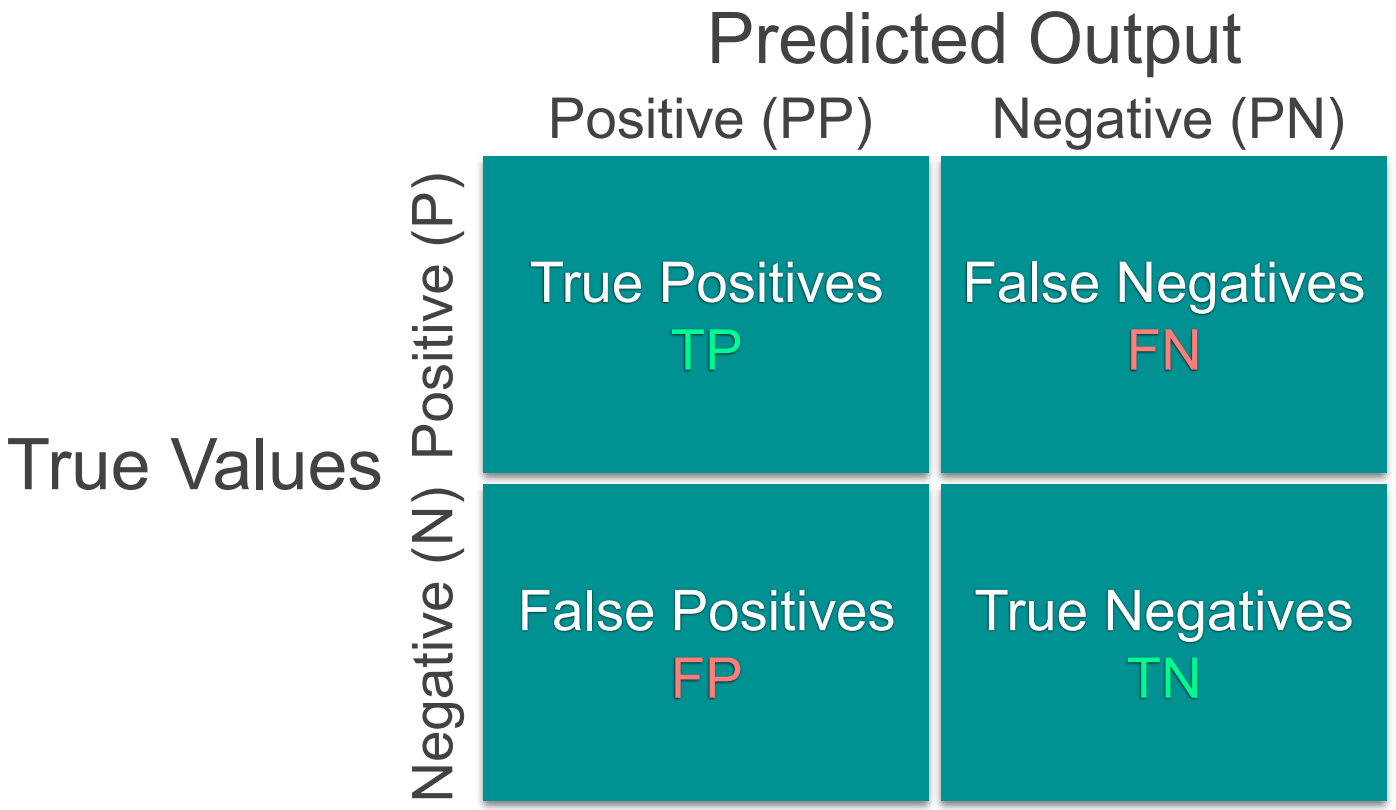
- Classification predicts categorical outcomes or class probabilities.

$$\textbf{ErrorRate} = \frac{1}{n}\sum_{i=1}^{n} 1\{y_i \neq \hat{y}_i\}$$

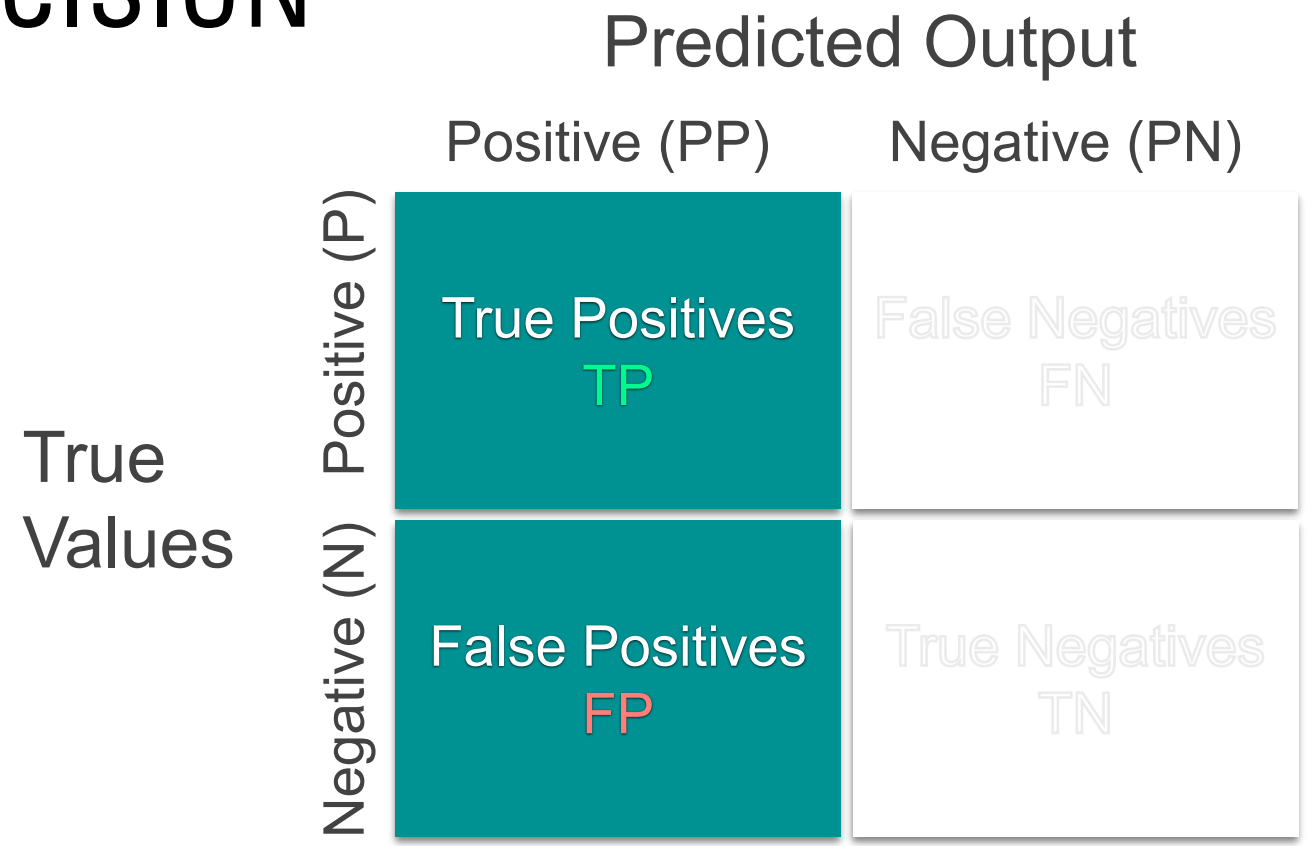
$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Total predictions}} = 1 - \text{Error rate}$$

- Both metrics treat all errors equally, regardless of their cost.
- With class imbalance, predicting only the majority class can yield high accuracy.

CONFUSION MATRIX



PRECISION

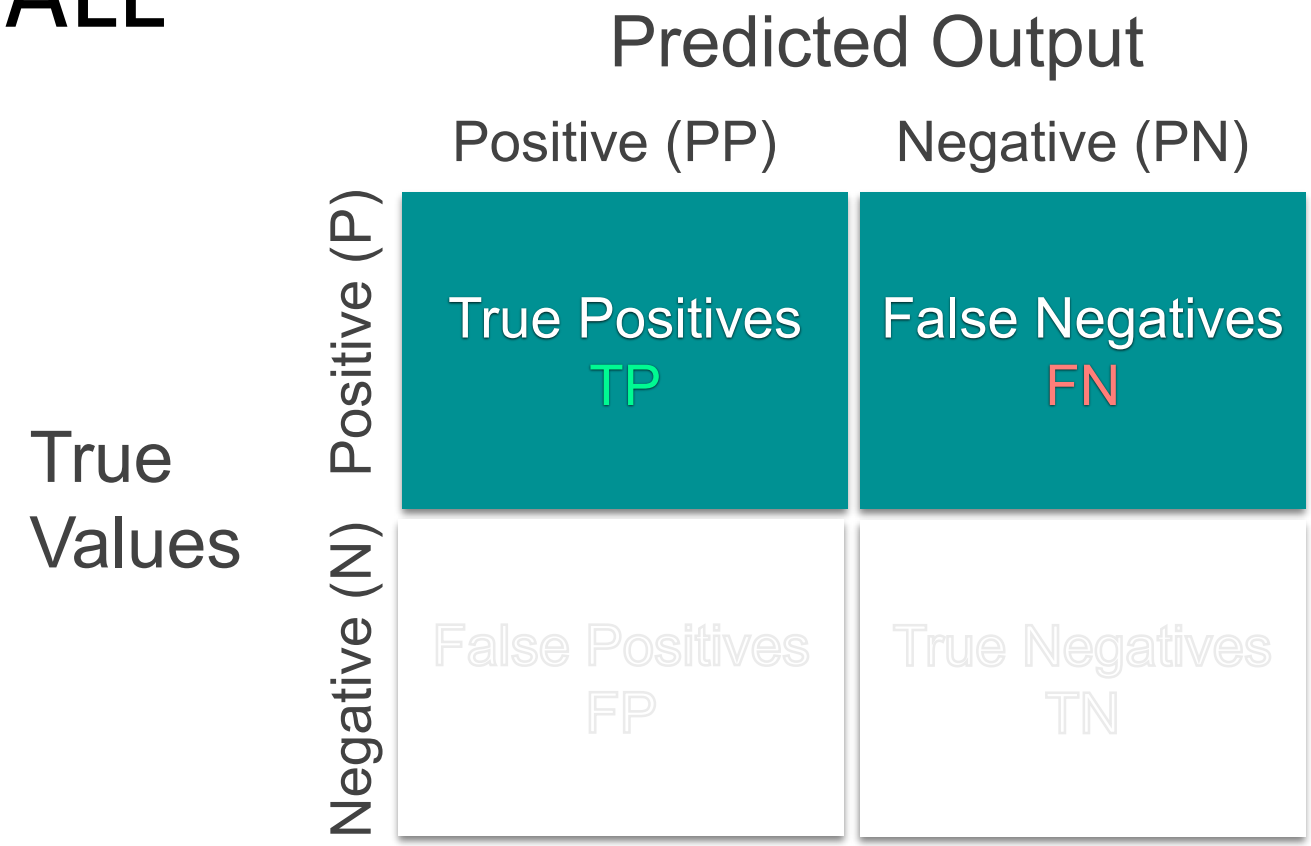


- Of those that are **predicted to be positive**, the percent that are actually positive
- Describes how good the model is at predicting the positive class
- Also call **positive predictive value**

$$Precision = \frac{TP}{TP + FP}$$

Under what conditions is high **precision** the most appropriate performance objective for a classification model?

RECALL



- Of those that are **actually positive**, the percent that are predicted to be positive
- The proportion of positives that are correctly identified
- Also called **sensitivity** or **true positive rate**

$$Recall = \frac{TP}{TP + FN}$$

Under what conditions is high **recall** the most appropriate performance objective for a classification model?

F1 SCORE

- The precision & recall are often combined into one metric called F_1
- F_1 is the **harmonic mean** of the precision and recall

$$F_1 = \frac{2}{\frac{1}{precision} + \frac{1}{recall}} = 2\left(\frac{precision \cdot recall}{precision + recall}\right)$$

  - The F_1 score favors classifiers with similar precision and recall scores

F_β SCORE

- The F_1 score can be generalized to give more weight to either the precision or recall

$$F_\beta = (1 + \beta^2)\left(\frac{precision \cdot recall}{(\beta^2 \cdot precision) + recall}\right)$$

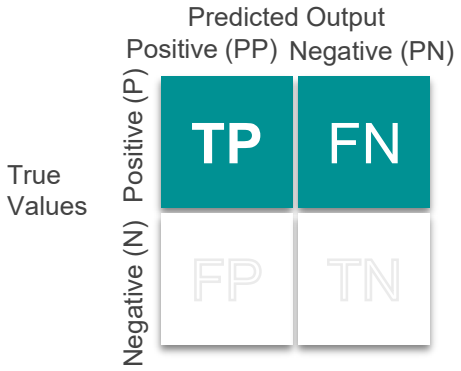
$0 < \beta < 1 \implies$ More weight given to the precision

$\beta > 1 \implies$ More weight given to the recall

$\beta = 1 \implies$ Equal weight given to precision and recall (F_1)

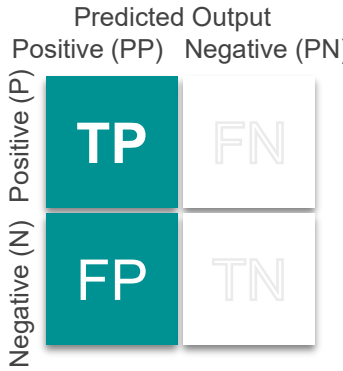
OTHER MEASURES

Recall / Sensitivity / True Positive Rate



$$\frac{TP}{TP + FN}$$

Precision / Positive Predictive Value

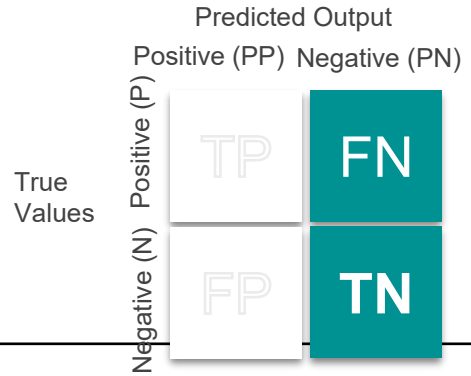


$$\frac{TP}{TP + FP}$$

F_1

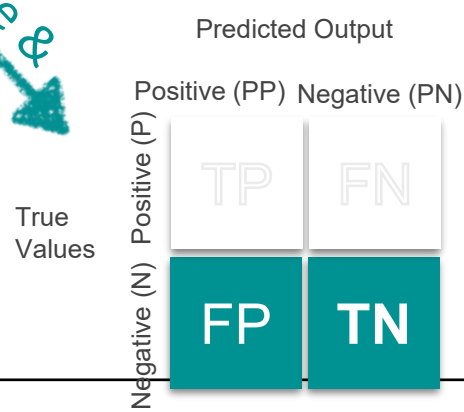
ROC Curve & AUC

Negative Predictive Value



$$\frac{TN}{TN + FN}$$

Specificity / True Negative Rate



$$\frac{TN}{TN + FP}$$

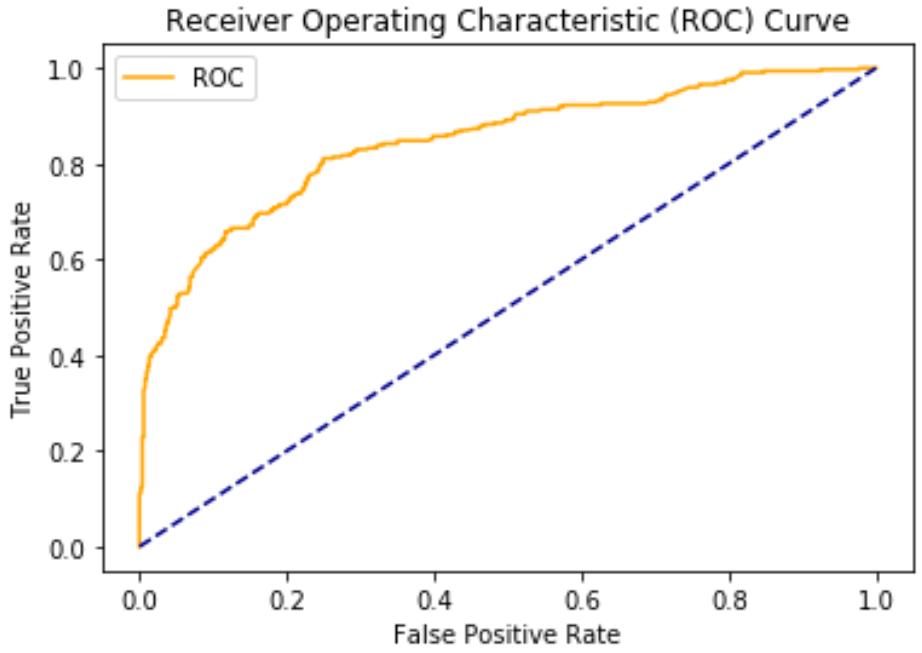
CLASSIFICATION THRESHOLDS

- A threshold converts a predicted probability or score into a class label.
  - Probability above the threshold: positive. At or below: negative.
- Example: A spam filter assigns an email P(spam) = 0.65.
  - Threshold 0.50: predict spam. Threshold 0.80: predict not spam.
- Lower thresholds predict more positives, with fewer misses but more false alarms.
- ROC curves compare true and false positive rates across thresholds.

ROC CURVE AND AUC

- A threshold converts a predicted probability or score into a class label.
  - Predict positive if $P(y_i = positive) > p$ $(0 < p < 1)$
- ROC plots true positive rate vs. false positive rate as the threshold varies.
  - True positive rate = sensitivity. False positive rate = 1 − specificity.
- AUC (area under the ROC curve) summarizes performance.
  - AUC = 1: perfect model. AUC = 0.5: random performance.

EXAMPLE ROC CURVE

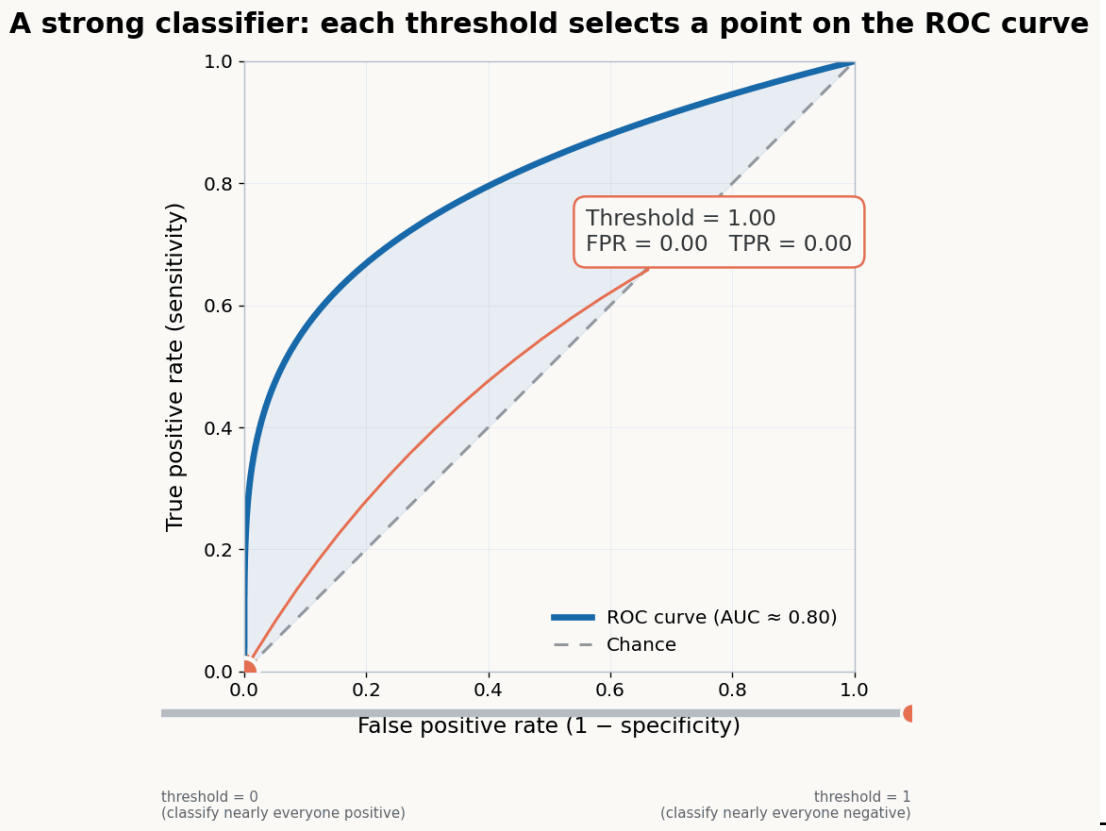


TPR = sensitivity. FPR = 1 − specificity.

AUC (area under the ROC curve) summarizes performance. 1 = perfect, 0.5 = random.

- **Each point uses a threshold.** Predict positive when the score exceeds it.
- **Lower threshold:** More positives. TPR and FPR cannot decrease. Move up/right toward (1, 1).
- **Higher threshold:** Fewer positives. TPR and FPR cannot increase. Move down/left toward (0, 0).
- **Tradeoff:** Lower thresholds miss fewer positives but raise false alarms. Higher thresholds do the reverse.

EXAMPLE ROC CURVE



THRESHOLDS AND ROC

Example: same ranking, different probabilities. Predict positive if P(+) > t.
Compare and contrast these models.

A. Scores near 0.5

Case	True class	P(+)
1	+	0.52
2	+	0.51
3	–	0.49
4	–	0.48

B. Nearly binary

Case	True class	P(+)
1	+	0.95
2	+	0.90
3	–	0.10
4	–	0.05

METRICS IN SCIKIT-LEARN

`.predict()` VS `.predict_proba()`

- For classification estimators, there are two ways to product predictions
- `.predict()` will produce an 1D array with n rows with the class prediction
    - Class with the highest probability
- `.predict_proba()` will produce a 2D array with n rows and n-class columns
    - Each column is the probability of the row observation being in class 0 to n-class

FUNCTION NAMES

https://scikit-learn.org/stable/api/sklearn.metrics.html#

- `accuracy_score(y_true, y_pred)`
  (default for many functions)
- `classification_report(y_true, y_pred)`
- `confusion_matrix(y_true, y_pred)`
- `f1_score(y_true, y_pred)`
- `precision_score(y_true, y_pred)`
- `recall_score(y_true, y_pred)`
- `roc_auc_score(y_true, y_score)`
  - Note: The second argument must be a score (e.g. probability) and not a class prediction

CLASSIFICATION REPORT

- This special function will show precision, recall, f1, and accuracy for all class (treat each class as positive class)

```
print(classification_report(y_test, yhat))
```

✓ 0.0s

	precision	recall	f1-score	support
0	0.95	0.99	0.97	980
1	0.91	1.00	0.95	1135
2	0.98	0.91	0.95	1032
3	0.96	0.96	0.96	1010
4	0.97	0.94	0.95	982
5	0.96	0.95	0.96	892
6	0.96	0.98	0.97	958
7	0.95	0.94	0.94	1028
8	0.99	0.91	0.95	974
9	0.93	0.95	0.94	1009
accuracy			0.95	10000
macro avg	0.96	0.95	0.95	10000
weighted avg	0.95	0.95	0.95	10000

MULTI-CLASS CLASSIFICATION

MULTI-CLASS STRATEGIES

- Some classifiers (e.g., Naive Bayes, KNN) are capable of handling multiple classes directly
- Other classifiers (e.g., logistic regression) are binary classifiers
- For binary classifiers, there are two strategies for extending to multiple classes:
  - One-versus-all (OvA) (sometimes called One-versus-rest (OvR))
  - One-versus-one (OvO)

ONE VERSUS ALL (OVA)

- For each class, train a model that classify that class versus all the rest:
  - Classify **1** versus **not 1**
  - Classify **2** versus **not 2**
  - etc.
- Classify a value into the classier whose classifier outputs the highest score (e.g., predictive probabilities)

ONE VERSUS ONE (OVO)

- Train a binary classifier for every pair of classes
  - **1** vs **2**
  - **1** vs **3**
  - **2** vs **3**
  - **4** vs **5**
  - Etc.
- Classify value into the class that wins the most duels

OVA VERSUS OVO

- Suppose there are q classes
- OvO requires training q(q-1)/2 classifiers
  - For the MNIST example this means 10(10-1)/2 = 45 classifiers
  - However, each classifier needs only use a small subset of the data
- In general OvA is the preferred strategy (only trains q classifiers)
- But for classifiers that scale poorly with the size of the training set (e.g., SVM) OvO is preferred

SOFTMAX FUNCTION

- OvA and OvO combine binary classifiers to handle multiple classes.
- Softmax supports another approach: extend logistic regression to predict all classes together.
- It converts one real-valued score per class into a probability distribution.
- Softmax also serves as the output function in multi-class neural networks.

$$p(c_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

z_i is a raw score for the class (for example the class logit)

Probability of class c_i

Sum over all classes